University of Eloued
Faculty of Exact Sciences
Department of Computer science
For Second-year Master: IA & Data Science

University season: 2026/2027
Module: Big Data Analytics

TP02: How to Work with Big Data Files (+5GB)

Student: Bey Heyam

1. Introduction

In this lab I compare three ways of working with a CSV file that is bigger than the RAM of my computer. The file is `train.csv` from the Kaggle competition "New York City Taxi Fare Prediction". It is 5.31 GB and It has 55,423,856 rows and 8 columns. My computer has 7.8 GB of RAM, so the whole file cannot fit in memory.

In every test I compute the same two things: the mean of `fare_amount` over the whole file, and the mean of `fare_amount` for each value of `passenger_count`. Tasks 2 and 3 only read these two columns. I measured the time with `time.perf_counter()` and the memory of the Python process (and its child processes) with `psutil`, sampled every 0.1 second.

2. Task 1: The Traditional Failure (OOM Challenge)

First I tried the normal way: `pd.read_csv(path)` on the whole file, with no chunks and no column selection. A real out-of-memory crash can freeze Windows, so I ran the loading in a separate process. The main script reads the system memory with psutil every half second and kills the loading process if the RAM reaches 90%.

Measure	Result
Outcome	Killed by the guard
System RAM at the stop	90.4 %
Time before the stop	3.5 s
Peak memory of the process	373.1 MB
Result	None (file not loaded)

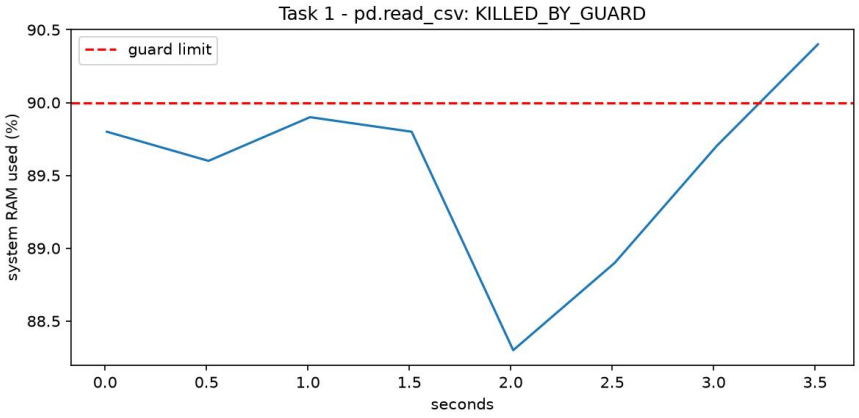


Figure 1: System RAM during Task 1 (the dashed line is the 90% limit)

The load did not finish. The guard stopped it after 3.5 seconds, when the system RAM went over 90%. I did not get a Python MemoryError, because my guard stopped the process before Python could raise it. Figure 1 also shows that the memory was already at about 88-90% when the test started, since Windows and my other programs were using most of the 7.8 GB.

This is why the process was stopped after allocating only 373 MB. So this run does not tell us how much memory pandas really needs. It only shows that there was almost no free memory for a 5 GB file.

A rough calculation shows why it cannot work anyway. There are 6 numeric columns of 8 bytes each, so 6 × 8 bytes × 55.4 million rows is about 2.7 GB. The two text columns (key and pickup_datetime) are stored as Python strings, which take much more space than in the file, and pandas also needs extra memory while it parses. The total is clearly more than 7.8 GB. This is an estimate and I did not measure it.

3. Task 2: Stream Processing (Pandas Chunking)

For the second task I used `pd.read_csv(..., chunksize=100000)`. This gives an iterator and each step returns a DataFrame of 100,000 rows, which is processed and then thrown away. With 55,423,856 rows this makes 555 chunks (554 full chunks and a smaller last one). I also read only the two needed columns, with the types float64 for fare_amount and int16 for passenger_count.

The main point is how to combine the chunks. The mean of the whole file is the total sum divided by the total number of rows:

$$mean = (S1 + S2 + ... + SK) / (n1 + n2 + ... + nK)$$

where Sk is the sum of fares in chunk k and nk is its number of rows. I cannot simply take the average of the chunk means, because the chunks do not always have the same size.

Example: chunk 1 has 3 rows with a mean of 10 (sum 30) and chunk 2 has 1 row with a mean of 20 (sum 20). The correct mean is (30 + 20) / (3 + 1) = 12.5. The average of the two means would give (10 + 20) / 2 = 15, which is wrong. The two results are the same only when all chunks have the same size.

So for each chunk I only add its sum and its count to running totals, and I do the division once at the end. For the mean per passenger_count I do the same with one sum and one count for each group. (To remember it: keep the sum and the count, not the mean.)

Measure	Result
Number of chunks	555
Rows processed	55,423,856
Sum of fare_amount	628,786,173.74
Mean fare_amount	11.345
Peak memory	110.9 MB
Execution time	60.4 s

The memory stays small because only the running totals are kept, so it depends on the size of a chunk and not on the size of the file. The time is 60.4 seconds because the chunks are read one after the other with a single core.

4. Task 3: Parallelized Out-of-Core Compute (Dask)

Lazy evaluation:

In Dask, `dd.read_csv()` and the operations after it (`sum`, `count`, `mean`, `groupby`) do not read or compute anything. Each call only adds a step to a task graph. In my run, building the graph took 0.066 seconds and the variables were Dask objects, not numbers. The data is read only when I call `dask.compute()`.

Because Dask sees the whole graph before running it, it can read only the two columns it needs, cut the file into partitions, process the partitions at the same time, and combine the partial results at the end so the full file is never in memory. I also computed the four results (sum, count, mean and the mean per group) in one single call, so the file is read only once.

Optimizing the graph

I used a block size of 128 MB, which gave 44 partitions, and I called `.optimize()` on the graph before the computation. The optimization should remove the unused columns and merge small steps into one. I ran the computation with the threaded scheduler.

Execution structure compared with Task 2

In Task 2 there is one loop: chunk 1, then chunk 2, and so on, on one core. In Dask the 44 partitions are independent tasks, so several of them can be processed at the same time by different threads. Only the last step, where the partial sums and counts are combined, is sequential.

Measure	Result
Number of partitions	44
Time to build the graph	0.066 s
Rows processed	55,423,856
Sum of fare_amount	628,786,173.74
Mean fare_amount	11.345
Peak memory	2,227.4 MB
Execution time	54.3 s

5. Performance Benchmark:

Method	Max RAM Utilization	Execution Speed
Standard Pandas	373 MB (system peak 90.4%)	N/A (stopped by the guard)
Pandas Chunking	111 MB	60.4 s (sequential)
Dask DataFrame	2,227 MB	54.3 s (multi-threaded)

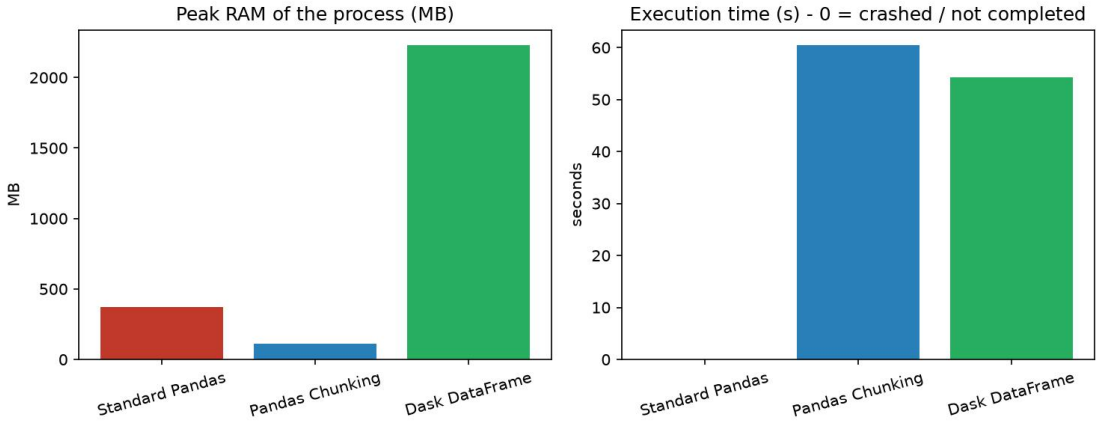


Figure 2: Peak memory and execution time of the three methods

Chunking and Dask gave exactly the same answers: the same number of rows, the same sum, the same global mean (11.345) and the same mean for every passenger_count. This is a good check that the sum-and-count method and the Dask reduction are equivalent. Dask returns the groups in a different order, but the values are identical.

passenger_count	Mean fare_amount	passenger_count	Mean fare_amount
0	9.015	7	31.789
1	11.216	8	29.981
2	11.838	9	36.993
3	11.541	34	13.300
4	11.766	49	2.500
5	11.208	51	9.300
6	12.126	129	8.900
		208	8.975

Table: mean fare_amount for each passenger_count (same result for Task 2 and Task 3)

The table also shows a problem with the data. A taxi has between 1 and 6 passengers, but the file contains values like 0, 7, 8, 9, 34, 49, 51, 129 and 208, which are certainly errors. The groups 7, 8 and 9 have a much higher mean fare (30 to 37) than the others (about 11 to 12), so they are probably abnormal records. These rows should be cleaned before using this column in a real analysis.

6. Discussion and Conclusion

Standard pandas cannot be used here. The file is 5.31 GB and the computer has 7.8 GB of RAM, and the loading was stopped after 3.5 seconds when the memory reached 90%.

Chunking was the best method for memory. It needed only 111 MB, which is about 20 times less than Dask (2,227 MB), and this number would stay the same for a bigger file. The price is that it is sequential and I had to write the sum and count logic myself.

Dask was the fastest, but only a little: 54.3 s against 60.4 s, so 60.4 / 54.3 = 1.11 times faster, or about 10% less time. It used a lot more memory because several partitions of 128 MB are loaded and parsed at the same time. I think the speed-up is small because reading a CSV file depends a lot on the disk, and because parsing with threads in Python is not fully parallel, but I did not test these two reasons. A smaller block size would probably reduce the memory of Dask.

The expected table in the lab sheet says that Dask should have a low and optimized memory use. In my results this is not the case on this computer: Dask used the most memory of the two methods that worked. The other expectations were correct: pandas fails, chunking has a low and constant memory, and Dask is faster than the sequential chunking.

In the end, both chunking and Dask let me process a file bigger than the RAM and gave the same exact result. With only 7.8 GB of RAM, chunking is the safer choice, and Dask is a good choice when memory is not a problem and a simpler pandas-like code is wanted.